Project Documentation: Implementation of an End-to-End BI Solution

From the Operational Database System (OLTP) to the Analytical Data Warehouse (DWH)

1. Project Roadmap

The main objective of this project was the transformation of unstructured sales data into a decision-relevant data structure for management purposes. The process was divided into three main layers:

• Raw Data Layer: Use of CSV files (Kaggle dataset) with over 500,000 transactions.
• Operational Layer (Retail_OLTP): A highly normalized relational database model (3NF) for precise daily data storage.
• Analytical Layer (Retail_DWH): A data warehouse in a star schema for high-performance queries and business reporting.

2. Description of Layers and Entities

A) Operational Database (Business DB - OLTP)

The design focused on data integrity and minimizing redundancy:

• Store & Employee: Management of 10 stores across 5 regions and 20 employees for clear responsibility assignment.
• Products & Customers: Centralized storage of master data to avoid duplicates.
• SalesHeaders & Details: Implementation of a master-detail model to separate general invoice information (date, customer) from specific line-item details (quantity, unit price).

B) Data Warehouse (DWH - OLAP)

Here, the structure was denormalized for analytical purposes and transformed into a star schema:

• Fact Table (Sales_Fact): The center of the model containing surrogate keys and measurable metrics (Quantity, UnitPrice, TotalPrice).
• Dimensions: Five key dimensions (Time, Product, Customer, Store, Employee) enabling multidimensional analysis.

3. Technical Challenges & Engineering Solutions (Lessons Learned)

Challenge 1: Categorization of Unstructured Data (Mapping)

• Problem: The raw data lacked a “product category” column, which is essential for management reporting.
• Solution: Creation of a mapping logic in Python. Based on keywords in the Description column (e.g., “Bag” or “Kitchen”), products were assigned to logical categories.

Challenge 2: Driver Errors and Date Formats (Data Type Mismatch)

• Problem: During data transfer via ODBC, the HYC00 error occurred because complex Python datetime formats could not be directly converted into SQL.
• Solution: In the ETL script, all timestamps were converted into standard strings using the .strftime() method. Additionally, the Hour field was extracted separately to enable hourly sales analysis.

Challenge 3: Price History Tracking (SCD Type 2)

• Problem: Price changes in the source system would distort historical financial reports (retroactive errors).
• Solution: Implementation of Slowly Changing Dimension (SCD) Type 2 in the Product_Dim table. Through the fields IsActive, Start_Date, and End_Date, the full price history remains preserved.

Challenge 4: Performance Optimization (Storage & Speed)

• Problem: Calculating TotalPrice during query execution across more than 500,000 records slowed down the system.
• Solution: Definition of TotalPrice as a Persisted Computed Column. The result is calculated once during loading and physically stored, significantly improving read performance.

4. Change Management Strategy (SCD Strategy)

• SCD Type 1: Applied to Store and Employee; only the current state is required here (e.g., an employee's current location).
• SCD Type 2: Applied to Product; indispensable for financial analysis to always associate sales with the price valid at the transaction time.

5. Conclusion and Highlights for the Presentation

1. Data Quality: Unstructured CSV data was transformed into a consistent and cleaned star schema.
2. Business Intelligence: The system answers complex questions such as: "What is the most profitable sales hour in stores located in the Bavaria region?" in real time.
3. Automation: Through Python (Pandas & PyODBC), a robust ETL pipeline was created, enabling daily automated batch runs.

Recommendation for the Future: As data volume increases, data cleansing should always be performed in Python before SQL bulk inserts in order to avoid database locks.

Date: May 2026